



Instituto Tecnológico de Mazatlán



**Semáforo Vehicular y Peatonal con Máquina de Estados Finitos (FSM)
en Arduino UNO R4 WiFi**

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernandez Barrón

Integrantes:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

1. Introducción

Los semáforos son uno de los ejemplos clásicos para introducir el diseño de sistemas de control secuencial, ya que combinan temporización, prioridades y eventos externos dentro de un ciclo de operación bien definido. En la presente práctica se diseñó e implementó un semáforo vehicular y peatonal utilizando una placa Arduino UNO R4 WiFi, modelando su comportamiento mediante una máquina de estados finitos (FSM, por sus siglas en inglés).

El sistema controla tres LEDs para el tránsito vehicular (rojo, amarillo y verde) y dos LEDs para el cruce peatonal (rojo y verde), además de un pulsador que permite a un peatón solicitar el cruce. A diferencia de una implementación básica basada en la función `delay()`, en este proyecto toda la temporización se resolvió de forma no bloqueante utilizando la función `millis()`, lo que permite que el microcontrolador siga atendiendo el estado del pulsador en todo momento sin detener la ejecución del programa.

Este documento presenta el desarrollo de la práctica, el material utilizado, la evidencia fotográfica del circuito armado y funcionando, así como el análisis de los resultados obtenidos y las conclusiones individuales de cada integrante del equipo.

2. Objetivos

Objetivo general:

- Diseñar e implementar un sistema de semáforo vehicular y peatonal controlado por una máquina de estados finitos sobre una plataforma Arduino UNO R4 WiFi, utilizando temporización no bloqueante.

Objetivos específicos:

- Modelar el comportamiento del semáforo como una máquina de estados finitos, definiendo claramente los estados, las transiciones y las condiciones que las disparan.
- Implementar la temporización de cada estado utilizando `millis()`, evitando el uso de `delay()` para no bloquear la lectura de entradas.
- Incorporar un pulsador con antirrebote (debounce) por software para gestionar la solicitud peatonal como un evento asíncrono.
- Establecer reglas de seguridad y prioridad, de modo que la solicitud peatonal solo pueda armarse durante los estados de verde o amarillo vehicular y se atienda de inmediato al llegar al rojo.
- Armar el circuito físico en protoboard, verificar su funcionamiento y documentar el proceso mediante evidencia fotográfica y diagramas.

3. Herramientas y material utilizado

- Arduino UNO R4 WiFi

- Arduino IDE
- Protoboard
- LEDs: rojo, amarillo y verde (vehicular) y rojo/verde (peatonal)
- Resistencias limitadoras de corriente
- Pulsador (botón) para la solicitud peatonal
- Cables de conexión (jumpers)

4. Desarrollo

El comportamiento del semáforo se modeló mediante un enum class con cuatro estados: VEH_VERDE, VEH_AMARILLO, VEH_ROJO y PEA_VERDE. Cada estado tiene asociada una acción de entrada (encendido de los LEDs correspondientes) y una duración fija, con excepción del estado VEH_ROJO, cuya salida hacia PEA_VERDE depende de si existe una solicitud peatonal pendiente.

El pulsador se leyó con un esquema de antirrebote por software que separa la lectura cruda del pin del estado ya confirmado, evitando así detecciones falsas por rebote mecánico del botón. La solicitud peatonal solo se "arma" (activa la bandera solicitudPeatonal) si el semáforo vehicular se encuentra en verde o amarillo; si el peatón presiona el botón en cualquier otro momento, la pulsación se ignora, ya que la solicitud se atenderá automáticamente en cuanto el vehicular llegue a rojo. Si nadie solicita el cruce, el ciclo vehicular continúa de forma autónoma.

El diagrama esquemático del circuito, elaborado en Tinkercad, se muestra en la Figura 1.

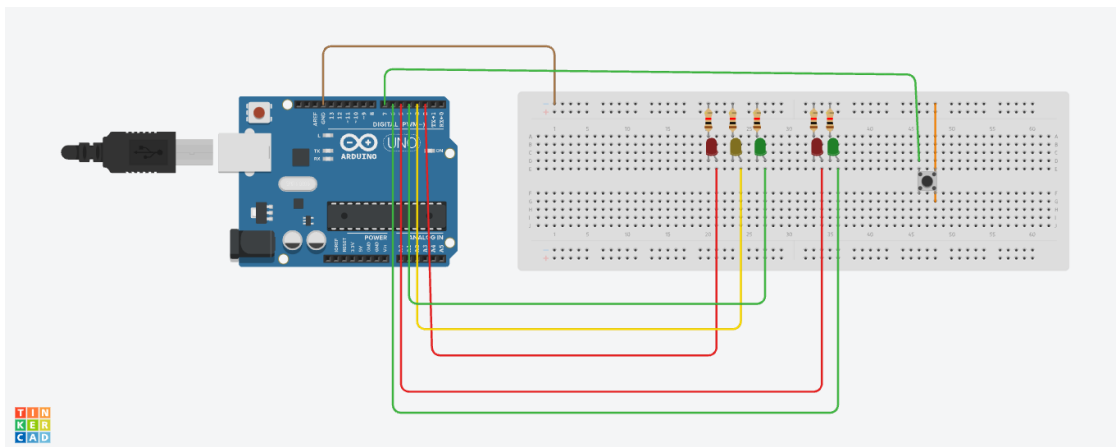


Figura 1. Diagrama esquemático del circuito (Tinkercad): Arduino UNO, LEDs vehiculares y peatonales, y pulsador.

5. Evidencia del circuito funcionando

A continuación se presenta la fotografía del circuito físico armado en protoboard, conectado a la placa Arduino UNO R4 WiFi y en funcionamiento durante las pruebas de la práctica.

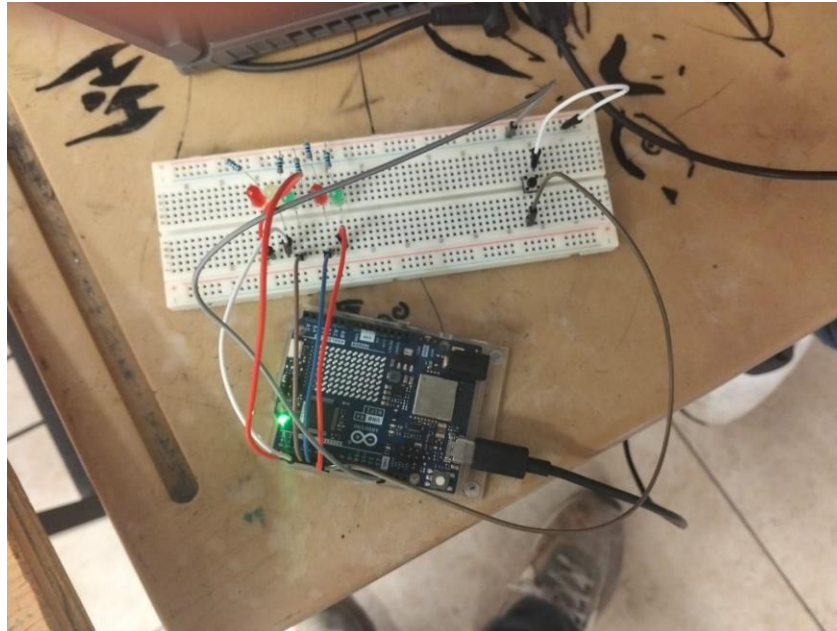


Figura 2. Circuito armado en protoboard con Arduino UNO R4 WiFi, LEDs vehiculares/peatonales y pulsador.

6. Análisis de resultados

Durante las pruebas se verificó que el ciclo vehicular se comporta según lo esperado: el LED verde permanece encendido durante 8 segundos, seguido de 2 segundos en amarillo y 6 segundos en rojo, mientras que el semáforo peatonal se mantiene en rojo durante todo este ciclo cuando no existe solicitud de cruce. Esto confirma que la lógica de temporización basada en `millis()` funciona correctamente y que el sistema no queda bloqueado en ningún momento, ya que el pulsador puede leerse en cada iteración del ciclo `loop()` sin interferencia de las esperas.

Se comprobó también la regla de prioridad y seguridad: al presionar el botón mientras el vehicular estaba en verde o amarillo, la bandera de solicitud se activó correctamente y el sistema transicionó a `PEA_VERDE` de forma inmediata en cuanto el semáforo vehicular llegó a rojo, sin esperar a que se agotara el tiempo completo de rojo. En cambio, al presionar el botón mientras el vehicular ya estaba en rojo o durante el cruce peatonal, la pulsación fue ignorada, tal como se especificó en el diseño, evitando así solicitudes redundantes o conflictivas.

El mecanismo de antirrebote demostró ser efectivo: no se detectaron activaciones múltiples o falsas a partir de una sola pulsación física del botón, gracias a la ventana de estabilización de 50 ms implementada en la función `botonPresionado()`. En conjunto, los resultados obtenidos coinciden con el comportamiento esperado de la máquina de estados finitos diseñada, validando tanto el hardware armado en protoboard como el firmware desarrollado.

7. Conclusiones individuales

Javier

Esta práctica permitió aplicar el patrón de máquina de estados finitos a un problema real de control secuencial, combinando temporización no bloqueante con `millis()` y eventos externos asíncronos (el botón peatonal). Se reforzó la separación de responsabilidades entre lectura de entradas, lógica de transición y aplicación de salidas, así como el diseño de reglas de seguridad y prioridad dentro de una FSM.

Gabriel

El desarrollo de esta práctica me permitió comprender la importancia de evitar el uso de `delay()` en sistemas donde se requiere atender entradas externas en todo momento, como el pulsador peatonal. Trabajar con `millis()` implicó pensar el programa como una serie de comparaciones de tiempo transcurrido en lugar de pausas fijas, lo cual al principio resultó más complejo, pero al final ofrece un control mucho más flexible y realista del comportamiento del semáforo. Además, el armado físico del circuito en protoboard ayudó a conectar la teoría de la máquina de estados con un resultado tangible y observable.

Rosa

Considero que el mayor aprendizaje de esta práctica fue entender cómo el antirrebote por software resuelve un problema físico real del pulsador, evitando lecturas falsas que podrían generar solicitudes peatonales incorrectas. También quedó clara la importancia de definir con precisión en qué estados se permite "armar" una solicitud y en cuáles se debe ignorar, ya que esas reglas son las que garantizan que el sistema se comporte de forma segura y predecible. En general, la práctica fortaleció mis habilidades tanto en programación estructurada como en el ensamblaje de circuitos electrónicos.